

CS1010J Lecture #11

Simple Recursion

The mirrors

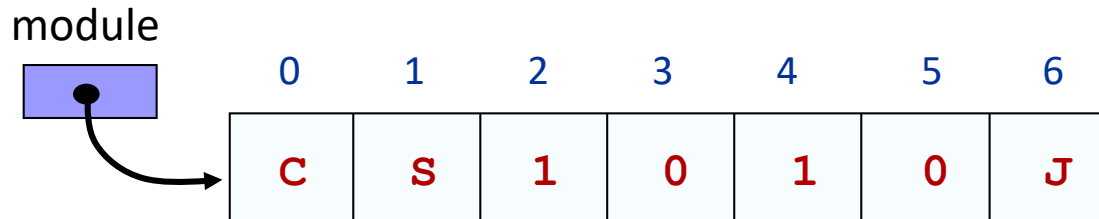


Department of Computer Science
School of Computing

Strings

- A **String** is an object whose data is a sequence of characters.

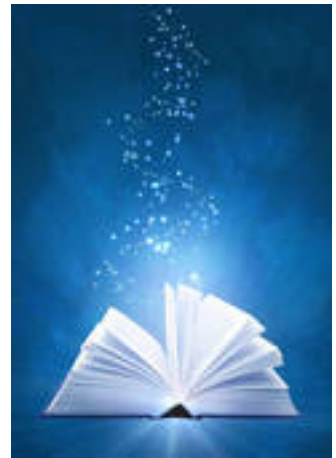
```
String module = "CS1010J";
```



- String input:
 - `scanner.next()`: read a single word (no space inside).
 - `scanner.nextLine()`: read a line of input.
- Java provides a rich set of **String** methods to use.

Learning Objectives

- At the end of this lecture, you should understand:
 - The nature of recursion
 - How to write recursive methods.



Pictorial Example

- This's a picture in a picture.



Filmic Example



```
void dream(int level) {  
    print("entered dream level" + level);  
    if (level == 3) {  
        wakeup(level);  
    } else {  
        dream(level+1);  
        wakeup(level);  
    }  
}
```

```
entered dream level 1  
entered dream level 2  
entered dream level 3  
waking up at level 3  
waking up at level 2  
waking up at level 1
```

```
void wakeup(int level) {  
    print("waking up at level " + level);  
}
```

Full story: <http://repeatgeek.com/technical/how-to-view-inception-through-code/>

Write Recursive Program

- There is **NO** new syntax needed for recursion.
- Recursion is a form of (algorithm) design; it is a problem-solving technique for divide-and-conquer paradigm.
 - A very important paradigm – many CS problems solved using it.
- Recursion is:

**A method where
the solution to a problem
depends on
solutions to smaller instances
of the SAME problem.**

Analogy

- Your friend is late for lecture...



How to Find Your Row?

- Get out of your seat, start counting from row 1
- Step down to the next row, +1 to count
- Repeat until you reach the first row

This is **iterative** thinking

How to Find Your Row?

- Your row number is 1 more than the row in front of you.
- Ask the person in front for his/her row number and add 1 to it.
- The person in front uses the same strategy.
- Eventually, person in front row simply replies 1.

This is recursive thinking

Demo #1 : Factorial (1/3)

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 2 \times 1$$

Iterative code (Ver. 1)

```
// Pre-condition: n >= 0
int factorialIter1(int n) {
    int ans = 1;
    for (int i = 1; i <= n; i++) {
        ans *= i;
    }
    return ans;
}
```

Iterative code (Ver. 2)

```
// Pre-condition: n >= 0
int factorialIter2(int n) {
    int ans = 1;
    while (n >= 1) {
        ans *= n;
        n--;
    }
    return ans;
}
```

Demo #1 : Factorial (2/3)

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 2 \times 1$$

recurrence relation:

$$n! = n \times (n - 1)!$$

$$0! = 1$$

- Doing it in a recursive way?

```
// Pre-condition: n >= 0
int factorial(int n) {
    if (n == 0) { // base case
        return 1;
    } else {
        return n * factorial(n-1);
    }
}
```

No loop structure at all!
But calling method
itself (recursively)
brings out repetition.

Side note: all the three versions work only for $n < 13$, due to the range of values permissible by the data type `int`. This is the limitation of the data type, not a limitation of the problem-solving model.

Demo #1 : Factorial (3/3)



■ Trace factorial(3)

** for simplicity, we write $f(3)$*

Winding:

$f(3)$: Since $3 \neq 0$, call $3 * f(2)$

$f(2)$: Since $2 \neq 0$, call $2 * f(1)$

$f(1)$: Since $1 \neq 0$, call $1 * f(0)$

$f(0)$: Since $0 == 0$, ...

Unwinding:

$f(0)$: Return **1**

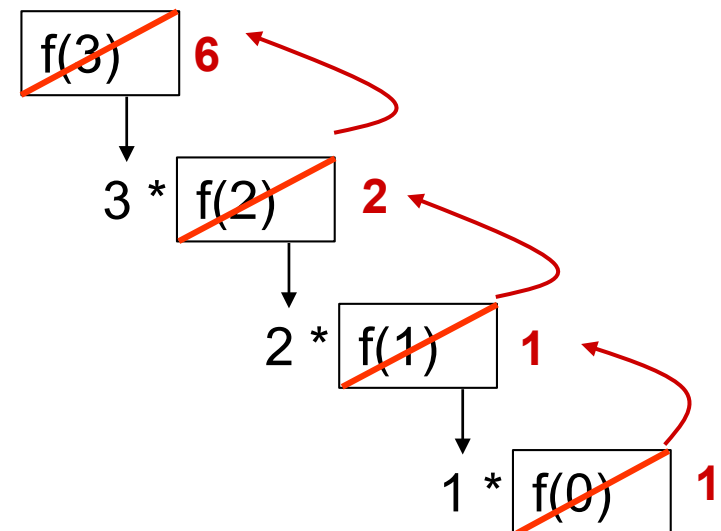
$f(1)$: Return $1 * f(0) = 1 * 1 = \mathbf{1}$

$f(2)$: Return $2 * f(1) = 2 * 1 = \mathbf{2}$

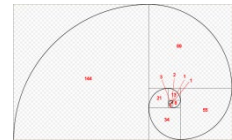
$f(3)$: Return $3 * f(2) = 3 * 2 = \mathbf{6}$

```
int f(int n) {  
    if (n == 0) { // base case  
        return 1;  
    } else {  
        return n * f(n-1);  
    }  
}
```

Trace tree:



Demo #2 : Fibonacci (1/3)



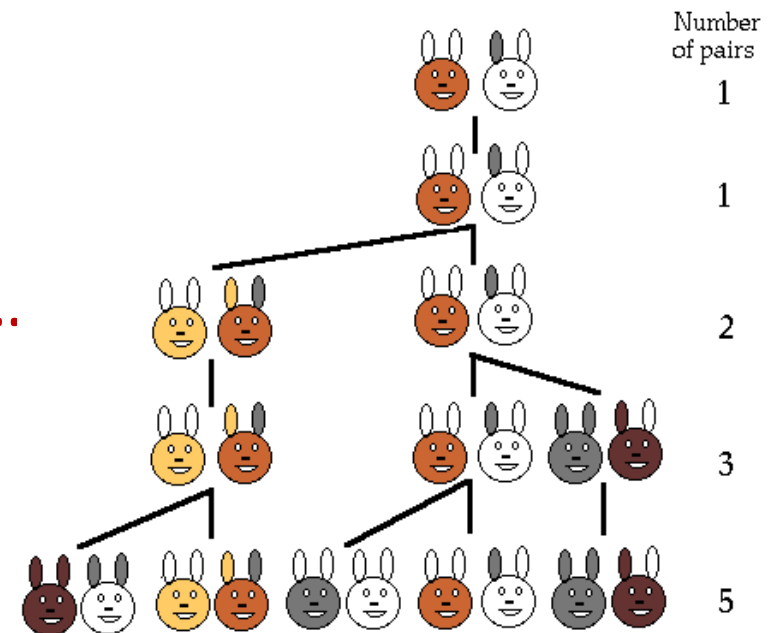
- The **Fibonacci series** models the rabbit population each time they mate:

1, 1, 2, 3, 5, 8, 13, 21, ...

- The modern version is:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

- Fibonacci numbers are found in nature (sea-shells, sunflowers, etc.)



□ <http://www.maths.surrey.ac.uk/hosted-sites/R.Knott/Fibonacci/fibnat.html>

Demo #2 : Fibonacci (2/3)

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Iterative version:

```
// Pre-condition: n >= 0
int fibIter(int n) {
    if (n < 2) {
        return n;
    }

    int prev1 = 1, prev2 = 0,
        current;

    for (int i = 2; i <= n; i++) {
        current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return current;
}
```

recurrence relation:

$$f_n = f_{n-1} + f_{n-2} \quad n \geq 2$$
$$f_0 = 0$$
$$f_1 = 1$$

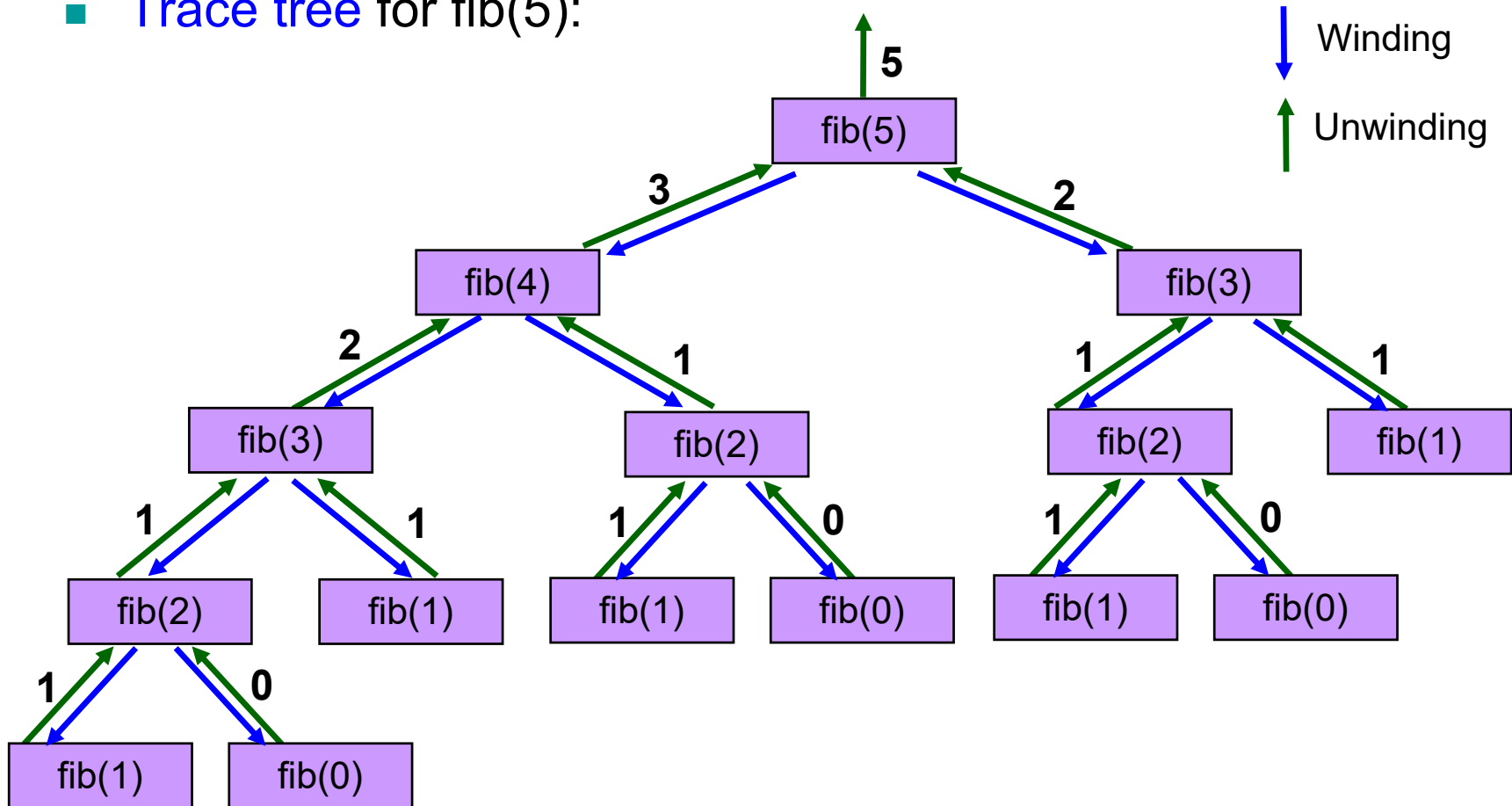
Recursive version:

```
// Pre-condition: n >= 0
int fib(int n) {
    if (n < 2) { // base case
        return n;
    } else {
        return fib(n-1) + fib(n-2);
    }
}
```

Demo #2 : Fibonacci (3/3)

```
int fib(int n) {  
    if (n < 2) { // base case  
        return n;  
    } else {  
        return fib(n-1) + fib(n-2);  
    }  
}
```

- $\text{fib}(n)$ makes 2 recursive calls: $\text{fib}(n-1)$ and $\text{fib}(n-2)$
- Trace tree for $\text{fib}(5)$:



Exercises #1 : Tracing

- Given the following 2 recursive methods, trace `mystery1(3902)` and `mystery2(3902)` using the trace tree method.

```
void mystery1(int n) {  
    if (n > 0) {  
        System.out.print(n%10);  
        mystery1(n/10);  
    }  
}
```

2093

```
void mystery2(int n) {  
    if (n > 0) {  
        mystery2(n/10);  
        System.out.print(n%10);  
    }  
}
```

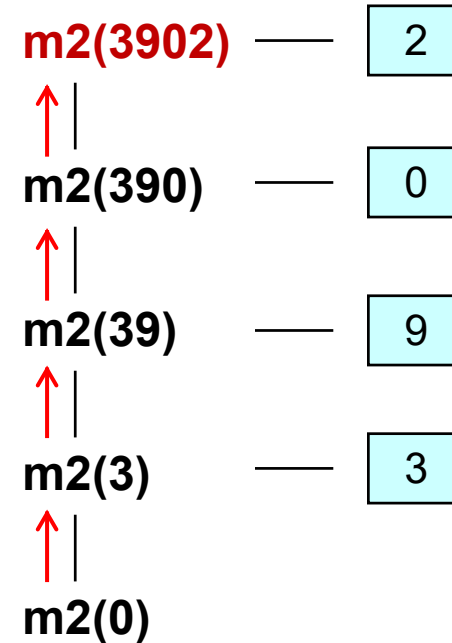
3902

The order of statements does matter!

Exercises #1 : Tracing

■ Trace tree of `mystery2(3902)`

```
void mystery2(int n) {  
    if (n > 0) {  
        mystery2(n/10);  
        System.out.print(n%10);  
    }  
}
```

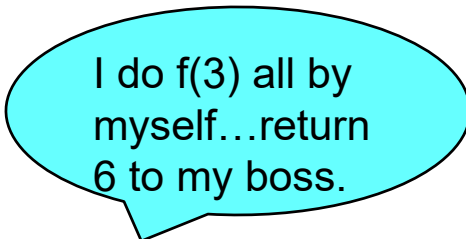


Gist of Recursion (1/6)

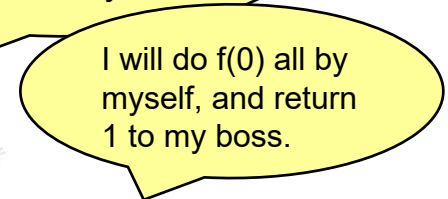
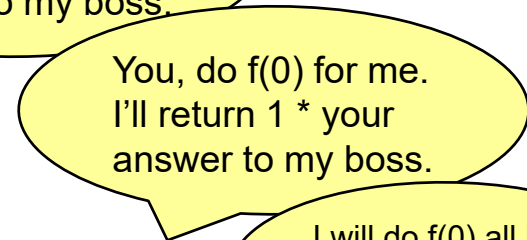
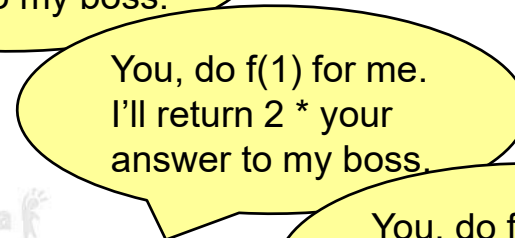
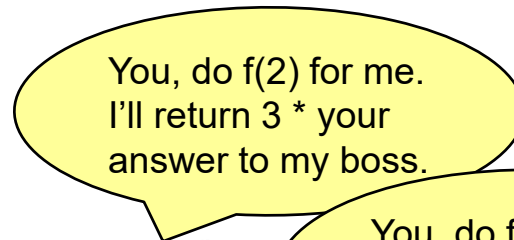
Iteration vs. Recursion: How to compute factorial(3)?



Iteration man

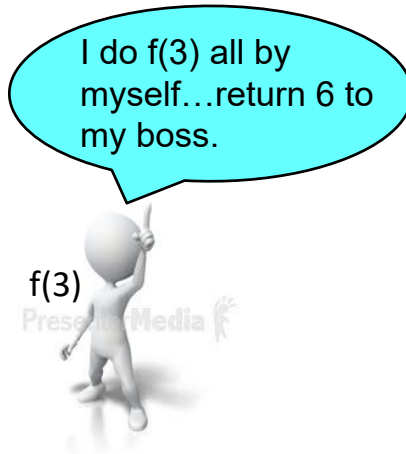


Recursion man

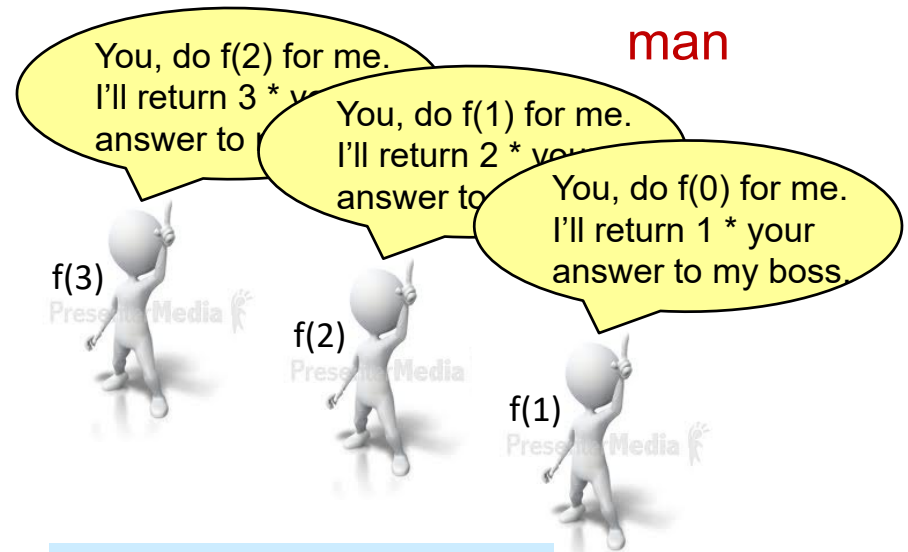


Gist of Recursion (2/6)

Iteration
man



Recursion
man



Iterative version:

```
// Pre-condition: n >= 0
int factorialIter1(int n) {
    int ans = 1, i;
    for (int i = 1; i <= n; i++) {
        ans *= i;
    }
    return ans;
}
```

Recursive version:

```
// Pre-condition: n >= 0
int factorial(int n) {
    if (n == 0) { // base case
        return 1;
    } else {
        return n * factorial(n-1);
    }
}
```

Gist of Recursion (3/6)

- Problems that lead themselves to a recursive solution have the following characteristics:
 - One or more **simple cases** (also called **base cases** or **anchor cases**) of the problem have a straightforward, non-recursive solution.
 - The other cases can be redefined in terms of problems that are smaller, i.e. **closer to the simple cases**.
 - By applying this redefinition process every time the recursive method is called, eventually the problem is reduced entirely to simple cases, which are easy to solve.
 - The solutions of the smaller problems are then combined to obtain the solution of the original problem.

Gist of Recursion (4/6)

- To write a recursive method:
 - Identify the **base case(s)** of the relation
 - Identify the **recurrence relation**

```
// Pre-condition: n >= 0
int factorial(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * factorial(n-1);
    }
}
```

```
// Pre-condition: n >= 0
int fib(int n) {
    if (n < 2) {
        return n;
    } else {
        return fib(n-1) + fib(n-2);
    }
}
```

Gist of Recursion (5/6)

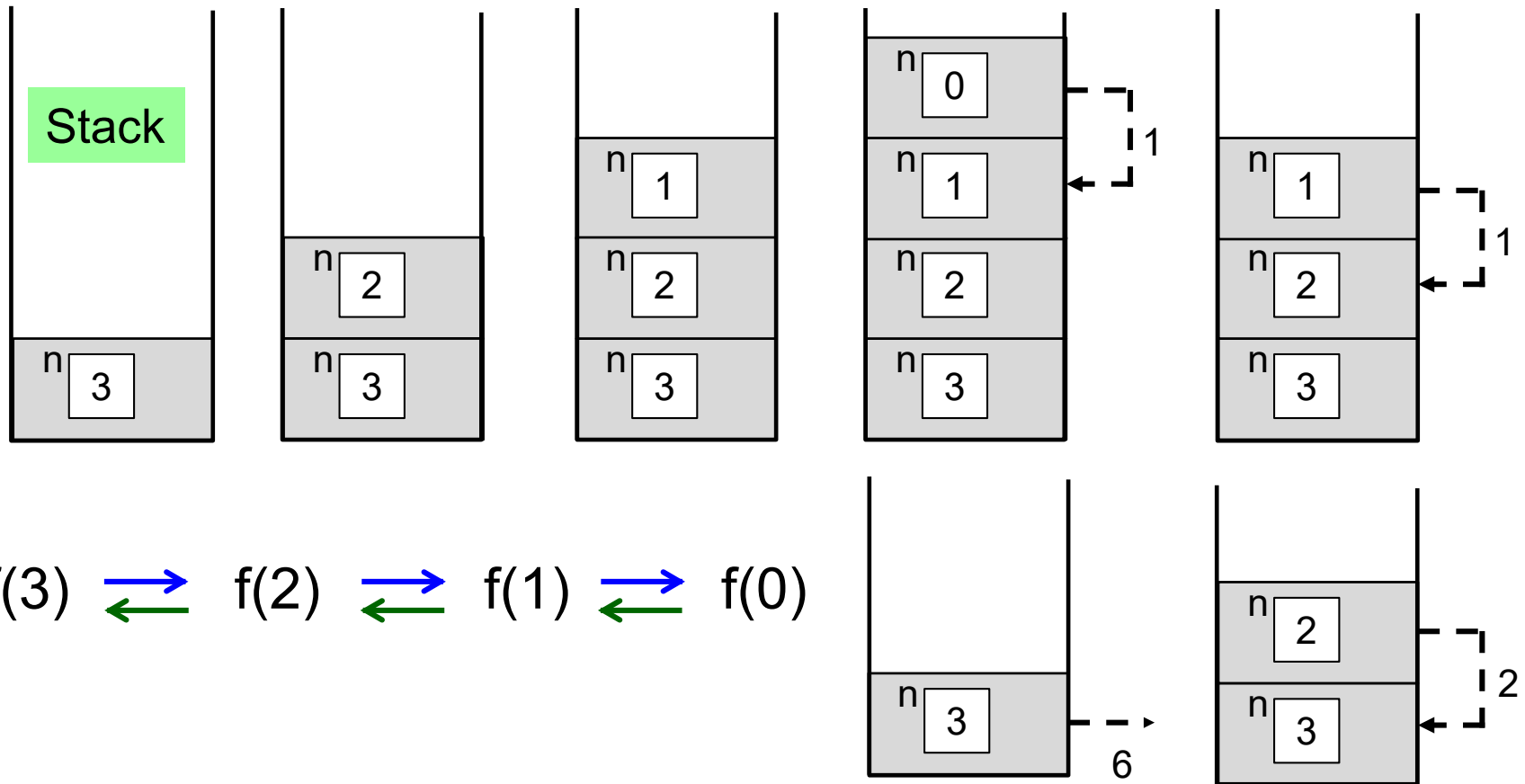
**Non-
examinable**

- When a method is called, an **activation record (frame)** is created by the OS.
 - Each activation record stores the local parameters and variables of the method and its return address.
 - Each record resides in the memory called stack.
 - Stack is also known as **LIFO** (last-in-first-out) structure.
- A recursive function can potentially create many activation records.
 - **Winding**: each recursive call creates a separate record.
 - **Unwinding**: each return to the caller erases its associated record.

Gist of Recursion (6/6)

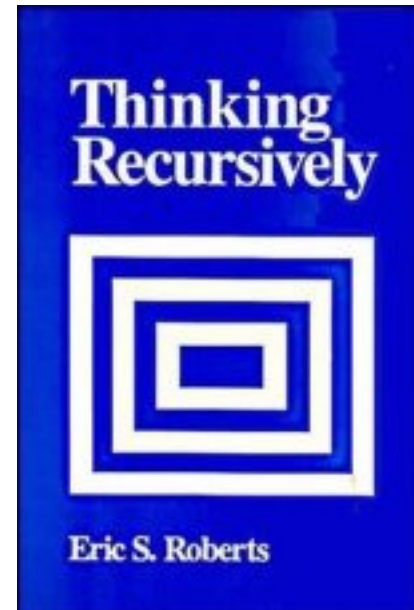
**Non-
examinable**

■ Example: factorial(3)



Thinking Recursively

- It is apparent that to do recursion you need to think “**recursively**”:
 - Breaking a problem into simpler problems that have identical form.



Demo #3 : Choosing Pokémon (1/3)

- How many ways can we choose k items out of n items?

Example: Choose 2 Pokémon out of 4



Choose 1 more

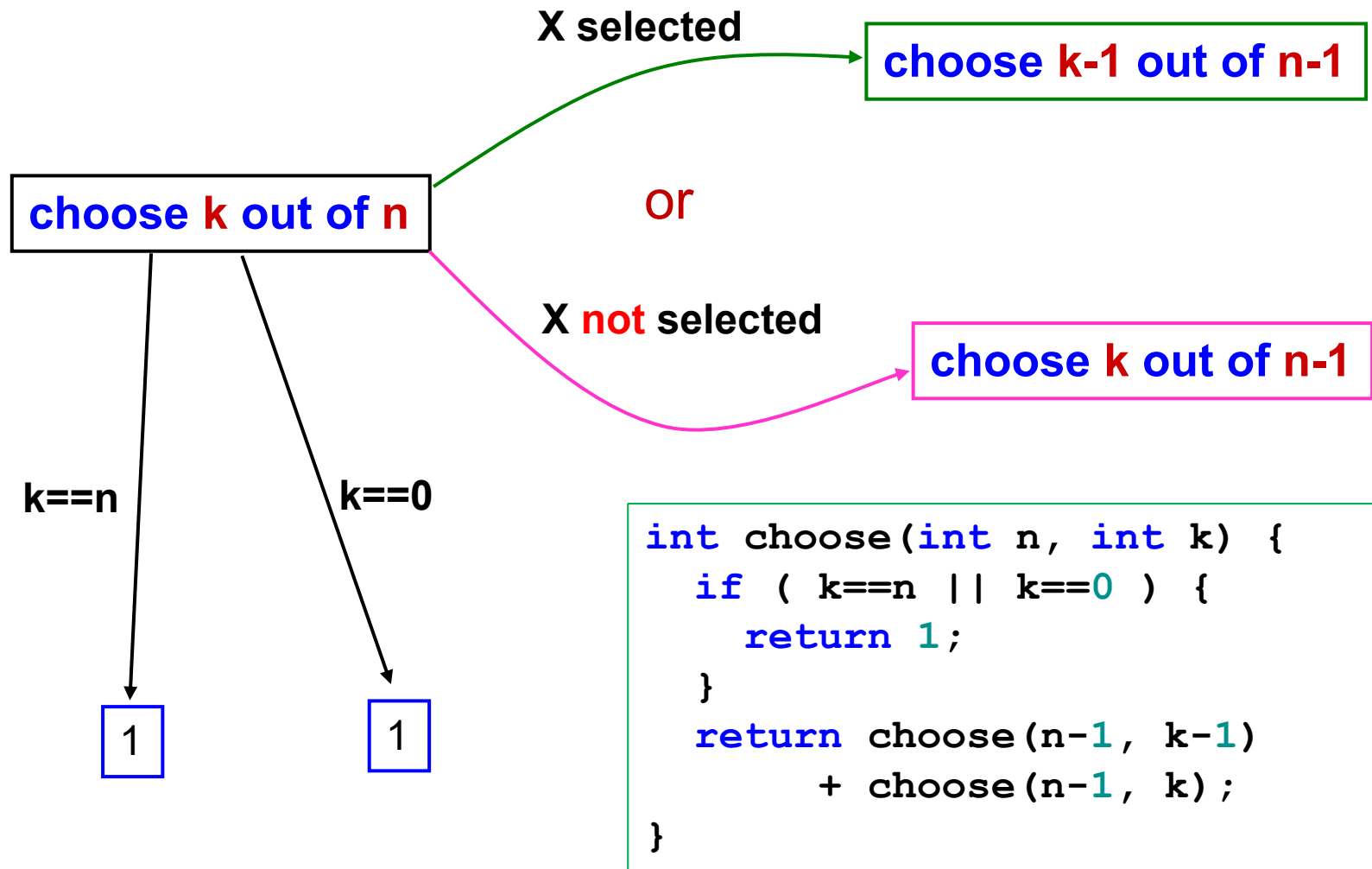
Either we choose this Pokémon,
then choose 1 out of remaining 3



Choose 2 more

Or we don't choose this Pokémon,
Then choose 2 out of remaining 3

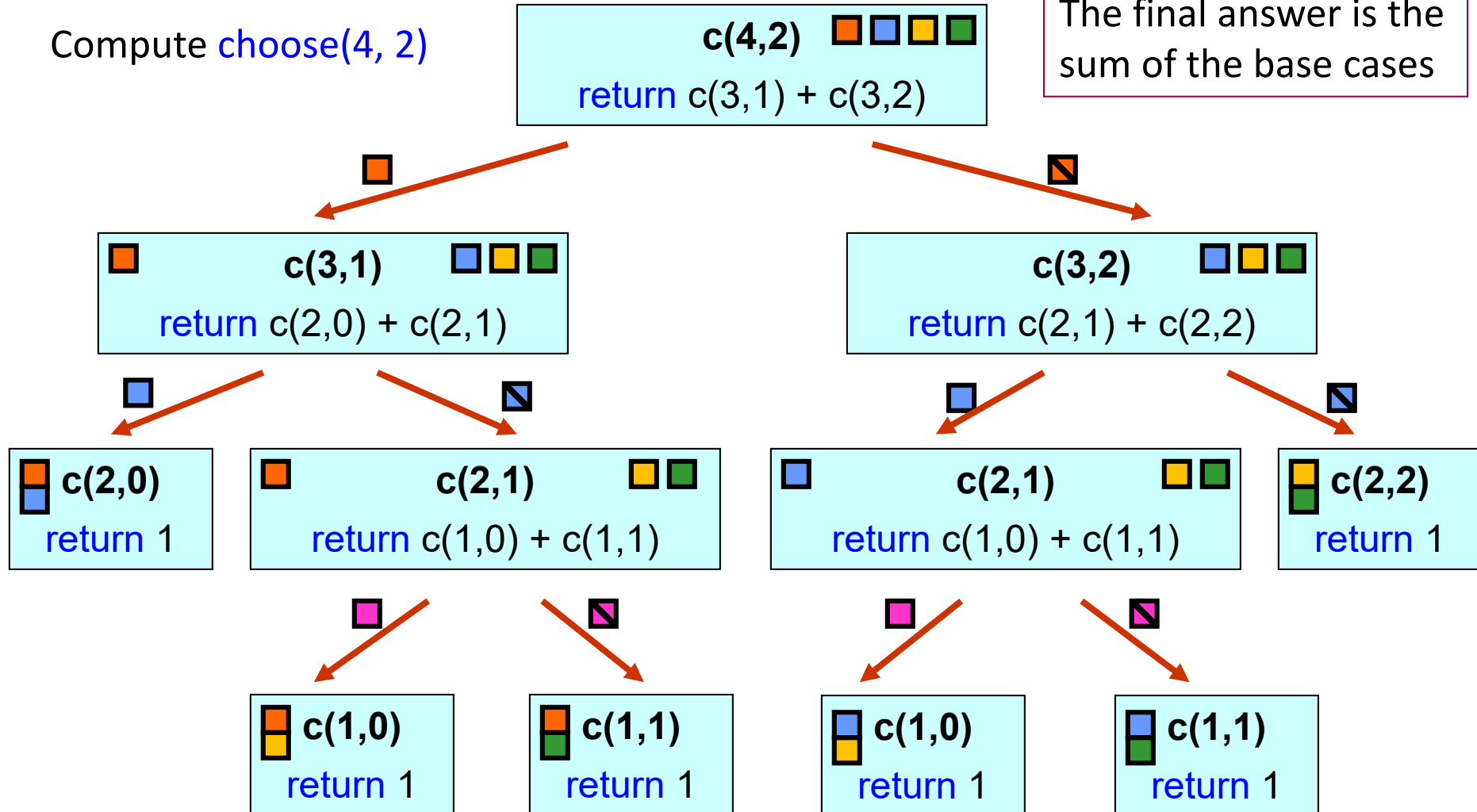
Demo #3 : Choosing Pokémon (2/3)



Demo #3 : Choosing Pokémon (3/3)

Compute `choose(4, 2)`

The final answer is the sum of the base cases

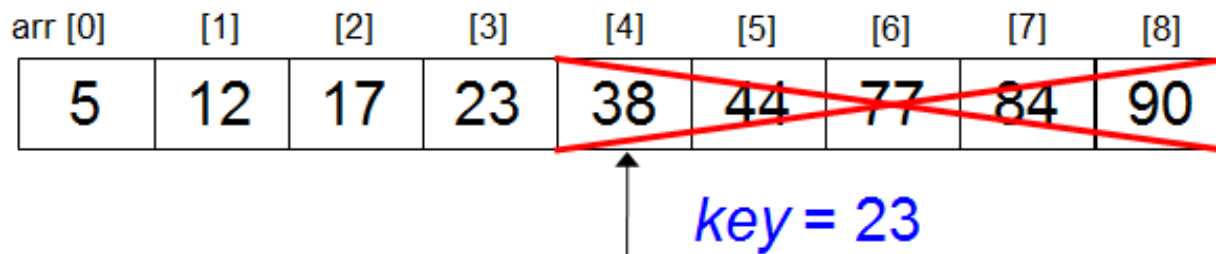


Demo #4 : Binary Search (1 / 3)

- Recursion can also be applied on data structures such as array.
 - The idea is to narrow down the scope of processing step by step.
- Example: [binary search](#)

arr [0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
5	12	17	23	38	44	77	84	90

↑
key = 23



Binary Search : Iterative Version

```
int binarySearch(int[] arr, int key) {  
  
    int low = 0, high = arr.length-1,;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        if (key == arr[mid]) {  
            return mid;  
        } else if (key < arr[mid]) {  
            high = mid - 1;  
        } else {  
            low = mid + 1;  
        }  
    }  
    return -1;  
}
```

Binary Search : Recursive Version

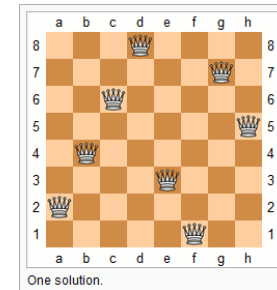
```
int binarySearchRec(int[] arr, int key,
                    int low, int high) {
    if (low > high) {
        return -1;
    } else {
        int mid = (low + high) / 2;
        if (key == arr[mid]) {
            return mid;
        } else if (key < arr[mid]) {
            return binarySearchRec(arr, key, low, mid-1);
        } else {
            return binarySearchRec(arr, key, mid+1, high);
        }
    }
}
```

Tracing Recursive Codes

- Beginners usually rely on tracing to understand the sequence of recursive calls and the passing back of results.
- However, tracing a recursive code is tedious and the trace tree could be huge (example: Fibonacci series).
- If you find that tracing is needed to aid your understanding, **start tracing with small problem sizes, then gradually see the relationship between the successive calls.**
- Students should **grow out of tracing habit** and understand recursion by examining the **relationship between the problem and its immediate sub-problem(s).**

Recursion versus Iteration

- Iteration can be more efficient.
 - Replaces method calls with looping
- Many problems are more naturally solved with recursion, which can provide elegant solutions.
 - Merge Sort (covered in CS2040)
 - The N Queens problem
- **Conclusion:** choice depends on the problem and the solution context. In general, use recursion if
 - A recursive solution is natural and easy to understand.
 - A recursive solution does not result in excessive duplicate computation.
 - The equivalent iterative solution is too complex.



Today's Summary

Simple Recursion

Recursion as a design methodology

The components of a recursive code

Difference between Recursion and Iteration

